07 - zz - Spine Leaf Architecture - SOLUTION by Drew

Lesson 5 Solutions

Reference solutions for the Spine-Leaf BGP Fabric exercises.

Exercise 1: Deploy and Configure the Fabric

Step 1: Deploy the topology.

```
$ cd lessons/clab/05-spine-leaf-bgp
$ containerlab deploy -t topology/lab.clab.yml
```

Expected output: table showing 10 running containers (2 spines, 4 leaves, 4 hosts).

Step 2: Apply BGP configuration to all 6 routers via gNMIc.

```
$ cd gnmic
$ gnmic -a clab-spine-leaf-bgp-spine1:57400 set --request-file
configs/spine1-bgp.json
$ gnmic -a clab-spine-leaf-bgp-spine2:57400 set --request-file
configs/spine2-bgp.json
$ gnmic -a clab-spine-leaf-bgp-leaf1:57400 set --request-file configs/leaf1-
bgp.json
$ gnmic -a clab-spine-leaf-bgp-leaf2:57400 set --request-file configs/leaf2-
bgp.json
$ gnmic -a clab-spine-leaf-bgp-leaf3:57400 set --request-file configs/leaf3-
bgp.json
$ gnmic -a clab-spine-leaf-bgp-leaf4:57400 set --request-file configs/leaf4-
bgp.json
```

Each config creates three routing policies (`import-all`, `export-connected`, `export-bgp`), a `host-subnets` prefix-set, enables BGP with the appropriate AS number, peer-group, neighbors, and sets `multipath maximum-paths` for ECMP.

Step 3: Examine the config files.

The config files apply several important pieces:

- **Three routing policies:** `import-all` (accept everything from peers), `export-connected` (advertise connected host /24s filtered by the `host-subnets` prefix-set), and `export-bgp` (re-advertise BGP-learned routes). These are chained as `["export-connected", "export-bgp"]` -- `export-connected` uses `default-action: next-policy` so non-matching routes (like BGP routes) fall through to `export-bgp`.
- **Prefix-set `host-subnets`:** Matches `10.20.0.0/16 mask-length-range 24..24` -- only /24 host subnets pass through `export-connected`. The /31 fabric links are filtered out because they don't need to be in BGP (each router already knows its directly connected links).
- **Multipath:** `maximum-paths: 2` on leaves (2 equal-cost paths, one per spine) and `maximum-paths: 4` on spines. SR Linux defaults to `maximum-paths: 1`, which means no ECMP without this setting.

Step 4: BGP neighbors on a leaf (2 established sessions to spines).

```
A:leaf1# show network-instance default protocols bgp neighbor
+---------------+--------+--------+-----------+--------+--------+--------+
| Peer          | Group  | AS     | Admin     | Session| Rcv    | Active |
|               |        |        | State     | State  | Routes | Routes |
+===============+========+========+===========+========+========+========+
| 10.10.1.0     |spines  | 65000  | enable    |establis| 3      | 3      |
|               |        |        |           | hed    |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
| 10.10.2.0     |spines  | 65000  | enable    |establis| 3      | 3      |
|               |        |        |           | hed    |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
```

leaf1 has 2 established sessions -- one to each spine. Each spine advertises 3 routes (the host /24 subnets it learned from the other 3 leaves). leaf1 installs all 3 as active from each spine. With `maximum-paths: 2`, both paths are used for ECMP -- traffic to each remote host subnet is load-balanced across both spines.

Step 5: BGP neighbors on a spine (4 established sessions to leaves).

```
A:spine1# show network-instance default protocols bgp neighbor
+---------------+--------+--------+-----------+--------+--------+--------+
| Peer          | Group  | AS     | Admin     | Session| Rcv    | Active |
|               |        |        | State     | State  | Routes | Routes |
+===============+========+========+===========+========+========+========+
| 10.10.1.1     |leaves  | 65001  | enable    |establis| 1      | 1      |
|               |        |        |           | hed    |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
| 10.10.1.3     |leaves  | 65002  | enable    |establis| 1      | 1      |
|               |        |        |           | hed    |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
| 10.10.1.5     |leaves  | 65003  | enable    |establis| 1      | 1      |
|               |        |        |           | hed    |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
| 10.10.1.7     |leaves  | 65004  | enable    |establis| 1      | 1      |
|               |        |        |           | hed    |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
```

spine1 has 4 established sessions -- one to each leaf. Each leaf advertises 1 route: its host /24 subnet (the `host-subnets` prefix-set filters out /31 fabric links from `export-connected`). All routes are active because each leaf's subnet is unique.

Session count: The fabric has 8 total unique BGP sessions (4 leaves x 2 spines). Each leaf has 2 sessions (one per spine). Each spine has 4 sessions (one per leaf).

Step 6: Cross-leaf pings all succeed.

```
$ docker exec clab-spine-leaf-bgp-host1 ping -c 3 10.20.2.2
PING 10.20.2.2 (10.20.2.2): 56 data bytes
64 bytes from 10.20.2.2: seq=0 ttl=61 time=3.456 ms
64 bytes from 10.20.2.2: seq=1 ttl=61 time=2.123 ms
64 bytes from 10.20.2.2: seq=2 ttl=61 time=1.876 ms
--- 10.20.2.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

```
$ docker exec clab-spine-leaf-bgp-host1 ping -c 3 10.20.3.2
PING 10.20.3.2 (10.20.3.2): 56 data bytes
64 bytes from 10.20.3.2: seq=0 ttl=61 time=3.234 ms
64 bytes from 10.20.3.2: seq=1 ttl=61 time=1.987 ms
64 bytes from 10.20.3.2: seq=2 ttl=61 time=1.654 ms
--- 10.20.3.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

```
$ docker exec clab-spine-leaf-bgp-host1 ping -c 3 10.20.4.2
PING 10.20.4.2 (10.20.4.2): 56 data bytes
64 bytes from 10.20.4.2: seq=0 ttl=61 time=3.567 ms
64 bytes from 10.20.4.2: seq=1 ttl=61 time=2.345 ms
64 bytes from 10.20.4.2: seq=2 ttl=61 time=1.987 ms
--- 10.20.4.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

```
$ docker exec clab-spine-leaf-bgp-host2 ping -c 3 10.20.4.2
PING 10.20.4.2 (10.20.4.2): 56 data bytes
64 bytes from 10.20.4.2: seq=0 ttl=61 time=3.345 ms
64 bytes from 10.20.4.2: seq=1 ttl=61 time=2.012 ms
64 bytes from 10.20.4.2: seq=2 ttl=61 time=1.789 ms
--- 10.20.4.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

Notice TTL=61. Starting TTL is 64, and the packet crosses 3 routers (leaf1 -> spine -> leaf2), so 64 - 3 = 61. This confirms the Clos path: every cross-leaf packet traverses exactly one spine.

Exercise 2: Read the Fabric Routing Table -- Observe ECMP

Step 1: Verify multipath is configured.

```
A:leaf1# info network-instance default protocols bgp afi-safi ipv4-unicast
multipath
    network-instance default {
        protocols {
            bgp {
                afi-safi ipv4-unicast {
                    multipath {
                        maximum-paths 2
                    }
                }
            }
        }
    }
```

SR Linux defaults to `maximum-paths 1` -- BGP picks a single best path and installs only that one. With `maximum-paths 2`, BGP installs both equal-cost paths (one per spine) into the routing table, enabling ECMP load balancing.

Step 2: leaf1's routing table showing ECMP entries.

```
A:leaf1# show network-instance default route-table ipv4-unicast summary
+-----------------+-------+-----------+--------------------+----------+
|   Prefix        | Type  | Route-Owner| Next-hop          | Metric   |
+=================+=======+===========+====================+==========+
| 10.10.1.0/31    | local | net_inst   | ethernet-1/49.0    | 0        |
| 10.10.2.0/31    | local | net_inst   | ethernet-1/50.0    | 0        |
| 10.20.1.0/24    | local | net_inst   | ethernet-1/1.0     | 0        |
| 10.20.2.0/24    | bgp   | bgp_mgr    | 10.10.1.0          | 0        |
|                 |       |            | 10.10.2.0          |          |
| 10.20.3.0/24    | bgp   | bgp_mgr    | 10.10.1.0          | 0        |
|                 |       |            | 10.10.2.0          |          |
| 10.20.4.0/24    | bgp   | bgp_mgr    | 10.10.1.0          | 0        |
|                 |       |            | 10.10.2.0          |          |
+-----------------+-------+-----------+--------------------+----------+
```

The remote host subnets (10.20.2.0/24, 10.20.3.0/24, 10.20.4.0/24) each show **2 next-hops** -- one via spine1 (10.10.1.0) and one via spine2 (10.10.2.0). This is ECMP: equal-cost multipath. Both paths have the same AS path length (2 AS hops: spine AS then remote leaf AS), so BGP installs both as equal-cost alternatives. The summary shows `IPv4 prefixes with active ECMP routes: 3`.

Notice that no /31 fabric link prefixes appear in the BGP routes. The `host-subnets` prefix-set on `export-connected` matches only `10.20.0.0/16 mask-length-range 24..24`, so /31 links like 10.10.1.0/31 are filtered out of BGP advertisements. They only appear as `local` routes from direct connection.

Step 3: Traceroute from host1 to host4 (multiple runs).

```
$ docker exec clab-spine-leaf-bgp-host1 traceroute -n -w 2 10.20.4.2
traceroute to 10.20.4.2 (10.20.4.2), 30 hops max, 60 byte packets
 1  10.20.1.1  1.234 ms  0.567 ms  0.432 ms
 2  10.10.1.0  1.456 ms  1.123 ms  0.987 ms
 3  10.10.1.7  1.678 ms  1.345 ms  1.234 ms
 4  10.20.4.2  2.123 ms  1.567 ms  1.456 ms
```

```
$ docker exec clab-spine-leaf-bgp-host1 traceroute -n -w 2 10.20.4.2
traceroute to 10.20.4.2 (10.20.4.2), 30 hops max, 60 byte packets
 1  10.20.1.1  1.123 ms  0.456 ms  0.345 ms
 2  10.10.2.0  1.345 ms  1.012 ms  0.876 ms
 3  10.10.2.7  1.567 ms  1.234 ms  1.123 ms
 4  10.20.4.2  2.012 ms  1.456 ms  1.345 ms
```

The path is always 4 hops: host1 -> leaf1 -> spine -> leaf4 -> host4. But the spine in hop 2 may differ between runs. The first run goes through spine1 (10.10.1.0), the second through spine2 (10.10.2.0). ECMP hashes flows across the available spines for load distribution.

Step 4: Answers to the questions.

- **How many hops between any two hosts?** Always 4: host -> leaf -> spine -> leaf -> host. In a Clos fabric, every leaf-to-leaf path is exactly 2 router hops (leaf-spine-leaf). Adding the host endpoints makes it 4 total hops. This path symmetry is a defining property of Clos -- no host pair is closer or farther than any other.
- **How many equal-cost paths exist between any two leaves?** 2 -- one through each spine. In a symmetric Clos fabric, every leaf is exactly 2 hops from every other leaf. Traffic always traverses exactly one spine. With 2 spines, there are always 2 equal-cost paths.
- **What happens to bandwidth if you add a third spine?** Aggregate bandwidth between any two leaves increases by 50% (from 2 ECMP paths to 3). Each spine provides one additional path. This is the horizontal scaling property of Clos: adding spines increases bisectional bandwidth without changing the leaf tier. No existing device needs reconfiguration beyond adding a BGP neighbor for the new spine.

Exercise 3: Break/Fix -- Spine Failure (Fabric Resilience)

Step 1: Routing table BEFORE spine failure (2 ECMP next-hops).

```
A:leaf1# show network-instance default route-table ipv4-unicast summary
+-----------------+-------+-----------+--------------------+----------+
|   Prefix        | Type  | Route-Owner| Next-hop          | Metric   |
+=================+=======+============+====================+==========+
| 10.10.1.0/31    | local | net_inst   | ethernet-1/49.0    | 0        |
| 10.10.2.0/31    | local | net_inst   | ethernet-1/50.0    | 0        |
| 10.20.1.0/24    | local | net_inst   | ethernet-1/1.0     | 0        |
| 10.20.2.0/24    | bgp   | bgp_mgr    | 10.10.1.0          | 0        |
|                 |       |            | 10.10.2.0          |          |
| 10.20.3.0/24    | bgp   | bgp_mgr    | 10.10.1.0          | 0        |
|                 |       |            | 10.10.2.0          |          |
| 10.20.4.0/24    | bgp   | bgp_mgr    | 10.10.1.0          | 0        |
|                 |       |            | 10.10.2.0          |          |
+-----------------+-------+-----------+--------------------+----------+
```

Each remote host subnet has 2 ECMP next-hops: spine1 (10.10.1.0) and spine2 (10.10.2.0).

Step 2: Disable all spine1 interfaces.

```
A:spine1# enter candidate
set / interface ethernet-1/1 admin-state disable
set / interface ethernet-1/2 admin-state disable
set / interface ethernet-1/3 admin-state disable
set / interface ethernet-1/4 admin-state disable
commit now
```

Step 3: Continuous ping showing brief loss then recovery.

```
$ docker exec clab-spine-leaf-bgp-host1 ping -c 30 -i 1 10.20.4.2
PING 10.20.4.2 (10.20.4.2): 56 data bytes
64 bytes from 10.20.4.2: seq=0 ttl=61 time=2.345 ms
64 bytes from 10.20.4.2: seq=1 ttl=61 time=1.234 ms
64 bytes from 10.20.4.2: seq=2 ttl=61 time=1.123 ms

--- spine1 interfaces disabled here ---

(several packets lost while BGP converges)

64 bytes from 10.20.4.2: seq=8 ttl=61 time=3.456 ms
64 bytes from 10.20.4.2: seq=9 ttl=61 time=2.123 ms
...
64 bytes from 10.20.4.2: seq=29 ttl=61 time=1.567 ms
--- 10.20.4.2 ping statistics ---
30 packets transmitted, 25 packets received, 17% packet loss
```

Brief loss during convergence, then full recovery. All traffic now flows through spine2.

Step 4: Routing table AFTER spine failure (1 next-hop).

```
A:leaf1# show network-instance default route-table ipv4-unicast summary
+-----------------+-------+-----------+--------------------+----------+
|   Prefix        | Type  | Route-Owner| Next-hop          | Metric   |
+=================+=======+===========+====================+==========+
| 10.10.1.0/31    | local | net_inst  | ethernet-1/49.0    | 0        |
| 10.10.2.0/31    | local | net_inst  | ethernet-1/50.0    | 0        |
| 10.20.1.0/24    | local | net_inst  | ethernet-1/1.0     | 0        |
| 10.20.2.0/24    | bgp   | bgp_mgr   | 10.10.2.0          | 0        |
| 10.20.3.0/24    | bgp   | bgp_mgr   | 10.10.2.0          | 0        |
| 10.20.4.0/24    | bgp   | bgp_mgr   | 10.10.2.0          | 0        |
+-----------------+-------+-----------+--------------------+----------+
```

ECMP entries dropped from 2 next-hops to 1. All traffic now goes exclusively through spine2 (10.10.2.0). The spine1 next-hop (10.10.1.0) was withdrawn when the BGP session went down.

Step 5: BGP neighbors on leaf1 showing spine1 session down.

```
A:leaf1# show network-instance default protocols bgp neighbor
+---------------+--------+--------+-----------+--------+--------+--------+
| Peer          | Group  | AS     | Admin     | Session| Rcv    | Active |
|               |        |        | State     | State  | Routes | Routes |
+===============+========+========+===========+========+========+========+
| 10.10.1.0     |spines  | 65000  | enable    |active  | 0      | 0      |
|               |        |        |           |        |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
| 10.10.2.0     |spines  | 65000  | enable    |establis| 6      | 3      |
|               |        |        |           | hed    |        |        |
+---------------+--------+--------+-----------+--------+--------+--------+
```

The spine1 session dropped to `active` (trying to reconnect). The spine2 session remains `established` and carries all traffic.

Step 6: Traceroute -- all traffic through spine2.

```
$ docker exec clab-spine-leaf-bgp-host1 traceroute -n -w 2 10.20.4.2
traceroute to 10.20.4.2 (10.20.4.2), 30 hops max, 60 byte packets
 1  10.20.1.1  1.234 ms  0.567 ms  0.432 ms
 2  10.10.2.0  1.456 ms  1.123 ms  0.987 ms
 3  10.10.2.7  1.678 ms  1.345 ms  1.234 ms
 4  10.20.4.2  2.123 ms  1.567 ms  1.456 ms
```

Every run now shows spine2 (10.10.2.x) at hop 2. No ECMP variation -- there is only one path.

Step 7: Re-enable spine1 and verify ECMP returns.

```
A:spine1# enter candidate
set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/2 admin-state enable
set / interface ethernet-1/3 admin-state enable
set / interface ethernet-1/4 admin-state enable
commit now
```

After BGP converges (a few seconds), the routing table returns to 2 ECMP next-hops per remote subnet.

What happened and why it matters:

- **Capacity drops by 50% but zero connectivity is lost after convergence.** With 2 spines, losing one spine removes half the aggregate bandwidth (1/N per spine). But every leaf still reaches every other leaf through the remaining spine. The fabric's redundancy design means no single spine is a critical failure point.
- **In production, this scales linearly.** With 4 spines, losing one spine costs 25% bandwidth. With 8 spines, only 12.5%. The more spines, the less impact per failure.
- **Dual-homing (MLAG/ESI) extends this to leaf failures.** In a production fabric, hosts connect to two leaves (not one). If a leaf fails, the host's other uplink through the second leaf keeps it connected. Our lab uses single-homed hosts for simplicity, but the principle is the same: redundancy at every tier.

Exercise 4 (Challenge): Break/Fix -- Route Leak

Step 1: After injecting the rogue /25 prefix on leaf1, check leaf2's routing table.

```
A:leaf2# show network-instance default route-table ipv4-unicast summary
+-----------------+-------+-----------+--------------------+----------+
|   Prefix        | Type  | Route-Owner| Next-hop          | Metric   |
+=================+=======+===========+====================+==========+
| 10.10.1.2/31    | local | net_inst   | ethernet-1/49.0    | 0        |
| 10.10.2.2/31    | local | net_inst   | ethernet-1/50.0    | 0        |
| 10.20.1.0/24    | bgp   | bgp_mgr    | 10.10.1.2          | 0        |
|                 |       |            | 10.10.2.2          |          |
| 10.20.2.0/24    | local | net_inst   | ethernet-1/1.0     | 0        |
| 10.20.3.0/24    | bgp   | bgp_mgr    | 10.10.1.2          | 0        |
|                 |       |            | 10.10.2.2          |          |
| 10.20.4.0/24    | bgp   | bgp_mgr    | 10.10.1.2          | 0        |
|                 |       |            | 10.10.2.2          |          |
| 10.20.4.0/25    | bgp   | bgp_mgr    | 10.10.1.2          | 0        |
|                 |       |            | 10.10.2.2          |          |
+-----------------+-------+-----------+--------------------+----------+
```

The rogue entry is `10.20.4.0/25` -- a /25 prefix that covers the first half of leaf4's 10.20.4.0/24 host subnet. This prefix was advertised by leaf1 and propagated through both spines to all other leaves.

Step 2: Ping from host2 to host4 fails.

```
$ docker exec clab-spine-leaf-bgp-host2 ping -c 3 -W 5 10.20.4.2
PING 10.20.4.2 (10.20.4.2): 56 data bytes

--- 10.20.4.2 ping statistics ---
3 packets transmitted, 0 packets received, 100% packet loss
```

Step 3: Traceroute shows traffic heading toward leaf1 instead of leaf4.

```
$ docker exec clab-spine-leaf-bgp-host2 traceroute -n -w 2 10.20.4.2
traceroute to 10.20.4.2 (10.20.4.2), 30 hops max, 60 byte packets
 1  10.20.2.1  1.234 ms  0.567 ms  0.432 ms
 2  10.10.1.2  1.456 ms  1.123 ms  0.987 ms
 3  10.10.1.1  1.678 ms  1.345 ms  1.234 ms
 4  *  *  *
```

Traffic reaches leaf1 at hop 3, then disappears into the blackhole. leaf1 has a static route for 10.20.4.0/25 pointing at 192.0.2.1 (a nonexistent address), so the packet is dropped.

Why longest-prefix-match causes this: When the router looks up destination 10.20.4.2, it finds two matching prefixes:

- `10.20.4.0/24` (the legitimate route to leaf4) -- matches the first 24 bits
- `10.20.4.0/25` (the rogue route to leaf1's blackhole) -- matches the first 25 bits

The /25 is more specific (longer prefix) and wins. This is longest-prefix-match: routers always prefer the most specific route, regardless of where it came from. The /25 beats the /24 even though the /24 is the legitimate route pointing to the real destination.

This is how BGP hijacking works at internet scale. A malicious or misconfigured AS advertises a more-specific prefix covering someone else's address space. Every router in the path installs the more-specific route, diverting traffic to the hijacker. In 2008, Pakistan Telecom accidentally advertised a /25 covering YouTube's /24, taking YouTube offline for several hours worldwide. The mechanism is identical to what we just demonstrated.

Step 4: Fix -- remove the rogue advertisement on leaf1.

```
A:leaf1# enter candidate
delete / routing-policy prefix-set hijack
delete / routing-policy policy export-hijack
delete / network-instance default static-routes route 10.20.4.0/25
delete / network-instance default next-hop-groups group nhg-blackhole
set / network-instance default protocols bgp group spines export-policy
[export-connected export-bgp]
commit now
```

Step 5: Verify host4 is reachable again.

```
$ docker exec clab-spine-leaf-bgp-host2 ping -c 3 10.20.4.2
PING 10.20.4.2 (10.20.4.2): 56 data bytes
64 bytes from 10.20.4.2: seq=0 ttl=61 time=3.456 ms
64 bytes from 10.20.4.2: seq=1 ttl=61 time=2.123 ms
64 bytes from 10.20.4.2: seq=2 ttl=61 time=1.876 ms
--- 10.20.4.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

```
$ docker exec clab-spine-leaf-bgp-host3 ping -c 3 10.20.4.2
PING 10.20.4.2 (10.20.4.2): 56 data bytes
64 bytes from 10.20.4.2: seq=0 ttl=61 time=3.234 ms
64 bytes from 10.20.4.2: seq=1 ttl=61 time=1.987 ms
64 bytes from 10.20.4.2: seq=2 ttl=61 time=1.654 ms
--- 10.20.4.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

After removing the rogue prefix-set, export policy, static route, and blackhole next-hop group, leaf1 stops advertising the /25. The spines withdraw it from their tables, and all other leaves fall back to the legitimate /24 route pointing to leaf4. Connectivity is restored.

Key lesson: Longest-prefix-match is the fundamental forwarding rule in IP networking. A more-specific prefix always wins, regardless of the source. In a BGP fabric, this means route filtering and prefix validation are critical. Production fabrics use prefix-lists, RPKI, and max-prefix limits to prevent accidental or malicious route leaks from propagating.

Key Takeaways

1. **Clos spine-leaf architecture eliminates the hub bottleneck** -- every leaf connects to every spine, creating multiple equal-cost paths and removing single points of failure at the spine tier
2. **ECMP requires explicit multipath configuration** -- SR Linux defaults to `maximum-paths 1` (single best path). You must set `multipath maximum-paths` under the `ipv4-unicast` address family to enable load balancing across spines
3. **Prefix-set filters keep the routing table clean** -- only host /24 subnets belong in BGP; /31 fabric links are already known via direct connection and don't need to be advertised
4. **Fabric resilience degrades gracefully** -- losing a spine reduces aggregate bandwidth by 1/N but causes zero connectivity loss after convergence; the remaining spines absorb all traffic
5. **RFC 7938 eBGP is the data center standard** -- spines share a single ASN, each leaf gets its own, and every spine-leaf link is eBGP
6. **Path symmetry is a Clos property** -- every host pair is exactly 4 hops apart (host-leaf-spine-leaf-host), regardless of which leaves they connect to; this predictable latency simplifies application design
7. **Longest-prefix-match can be weaponized** -- a more-specific prefix hijacks traffic from a less-specific one, which is why production fabrics need strict route filtering and prefix validation